D02_Architecture_and_Data_Schema

D02 — Architecture and data schema

Branch: `cursor/d02-backend-cicd-38d2`
Scope: Backend, REST API, database (incubation funding deliverable D02)
Production: unchanged by this work (no production deploy)

1. Architecture

```
Mobile / Web client (Expo)
        │  HTTPS JSON  (Bearer JWT)
        ▼
Vercel Serverless entry
  api/v1/[...path].ts
        │
        ▼
lib/server/vercel-dispatch.ts
  resolves /api/v1/* → app/api/v1/**/route.ts handlers
        │
        ├── Auth: lib/server/auth-helpers.ts
        │     verifies Authorization: Bearer <JWT>
        │     via lib/server/app-jwt.ts (HS256, issuer "grpd")
        │     optional Supabase JWT via lib/server/supabase-jwt.ts
        │
        ├── AuthN issuance:
        │     POST /api/v1/auth/email/register
        │     POST /api/v1/auth/email/login
        │     POST /api/v1/auth/pin/login   (fixture allowlist for
CI/test)
        │     POST /api/v1/auth/logout
        │
        ├── AuthZ: lib/server/role-checks.ts
        │     club membership role/status →
view|join|invite|approve|manage_club
        │
        └── Persistence: Prisma Client → PostgreSQL
              prisma/schema.prisma + prisma/migrations/*
```

Request path (verified)

1. Client calls `/api/v1/...` with JSON body when needed.
2. Vercel rewrite (`vercel.json`) sends `/api/v1/:path*` → `api/v1/[...path].ts`.
3. Dispatcher loads the matching App Router-style handler under `app/api/v1/**`.
4. Protected handlers `await requireAuth(req)` — missing/invalid Bearer → `401 UNAUTHORIZED`.
5. Governance handlers also `await requireClubPermission(...)` — wrong role → `403 FORBIDDEN`.

Environments

Env	Database	API host	Notes
Dev / evidence (local)	Disposable PostgreSQL `grpd_test` on `127.0.0.1`	In-process dispatcher / local scripts	Synthetic `AUTH_JWT_SECRET` only
CI	GitHub Actions Postgres 14 service (`55439`)	Same code path via Jest	Workflow `.github/workflows/backend-verification.yml`
Production	Existing Neon / Vercel production	`grpd30.vercel.app`	**Not modified** by this D02 branch

2. Data schema (Prisma)

Models (after migration `20261031020000_add_app_users`)

Model	Table	Purpose
`AppUser`	`app_users`	Email/password accounts for self-serve auth (`email` unique, `passwordHash`)
`Club`	`clubs`	Club identity (`slug` unique), visibility, creator
`ClubMembership`	`club_memberships`	Unique `(userId, clubId)`; role + status + optional PR summary
`ClubInvite`	`club_invites`	Invite `code` unique, optional expiry
`Session`	`sessions`	Outing container; optional `clubId`; `visibility`; JSON `paceGroups`
`SessionAttendance`	`session_attendance`	Unique `(sessionId, userId)`; status incl. `suggested`/`left`;

Model	Table	Purpose
		`groupId`
`SessionTag`	`session_tags`	Per-session tagged person

Relationship sketch

```
AppUser (id) ──referenced by opaque userId strings──► ClubMembership / SessionAttendance / Session.hostUserId

Club 1──N ClubMembership
Club 1──N ClubInvite
Club 1──N Session (clubId nullable for autonomous public sessions)
Session 1──N SessionAttendance
Session 1──N SessionTag
```

Migrations applied on the D02 test database

1. `20250128000000_add_clubs_memberships`
2. `20250128100000_add_session_tags`
3. `20260130090000_add_membership_prs`
4. `20260130235238_add_session_fields`
5. `20260210022611_add_attendance_status_suggested_left`
6. `20261031020000_add_app_users` ← **added for D02 auth**

See `evidence/migrations.txt` for the live `prisma migrate deploy` transcript.

3. Security controls (D02)

- **No unwired `req.auth`:** network auth is Bearer JWT verification only (`lib/server/auth-helpers.ts`).
- **Issuance:** email register/login hash passwords with scrypt (`lib/server/password.ts`) and sign HS256 JWTs with `AUTH_JWT_SECRET`.
- **Role gate:** roster / invite / approve / assign require coach|admin (`manage_club` / `invite` / `approve_members`).
- **Pending members leak fixed:** `GET /api/v1/clubs/:id` returns `pendingMembers` only when the caller has `manage_club`.
- **Secrets:** never committed. Evidence artifacts redact JWTs and passwords.

4. Source anchors

- Schema: `prisma/schema.prisma`
- Migrations: `prisma/migrations/**`
- Dispatcher: `lib/server/vercel-dispatch.ts`, `api/v1/[...path].ts`
- Auth: `lib/server/auth-helpers.ts`, `lib/server/app-jwt.ts`, `app/api/v1/auth/**`
- Roles: `lib/server/role-checks.ts`
- Deploy adapter: `vercel.json`

- CI: `.github/workflows/backend-verification.yml`